

REFERENCE NOTES

Protocols & Behaviours

defprotocol · defimpl · @derive · impl_for
@callback · @behaviour · @impl · OTP behaviours

Elixir Training Programme

Elixir · protocols and behaviours

Reference notes · Instructor-led · Elixir 1.20.4 on Erlang/OTP 29

PROTOCOLS & BEHAVIOURS · REFERENCE NOTES

Two ways to write code that works for many things

Protocols let one function call work across many **data types** — the right implementation is picked at run time from the data. **Behaviours** let many **modules** promise the same set of functions — the compiler checks the promise. Part A covers protocols, Part B behaviours, and the last page puts them side by side.

Protocol → polymorphism over data types

- `defprotocol + defimpl ...`, `for: Type`
- Dispatch at **run time**, on the type of the first argument
- Missing implementation → `Protocol.UndefinedError` (or the fallback)

Behaviour → a contract between modules

- `@callback` in one module, `@behaviour` in another
- Checked at **compile time**
- Missing callback → a compiler **warning**

Contents

#	Topic	What it covers
A1	Protocols — core concepts	Protocol, <code>defimpl</code> , dispatch, fallback, built-ins
A2	Define, implement, use	The Size protocol for strings, maps, lists, tuples
A3	Fallback and structs	<code>for: Any</code> , <code>@fallback_to_any</code> , implementing for a struct
A4	Built-in protocols	<code>Enumerable</code> , <code>String.Chars</code> , <code>Inspect</code>
A5	Deriving and checking	<code>@derive</code> and <code>impl_for/1</code>
A6	Mini example: Blank	A practical protocol, plus the protocol takeaways
B1	Behaviours — core concepts	<code>@callback</code> , <code>@behaviour</code> , compiler check, optional callbacks
B2	Defining a behaviour	The Parser behaviour
B3	Reading a <code>@callback</code> line	<code>::</code> and <code>term()</code> , piece by piece, and more precise types
B4	Implementing and using	<code>JSONParser</code> , <code>CSVParser</code> , <code>ParserRunner</code> , <code>@impl</code>
B5	Worker and OTP behaviours	A worker contract; <code>GenServer</code> as a behaviour
B6	Checks, practices, recap	Runtime checks, best practices, behaviour vs protocol

HOW TO READ Every block marked **REAL OUTPUT** or **REAL WARNING** was produced on Elixir 1.20.4 / Erlang OTP 29, evaluating the code form by form the way iex does.

PART A

Protocols

Polymorphism over data types, with runtime dispatch

A1

Protocols — core concepts

The same function call, working for many types

Elixir protocols provide a way to achieve polymorphism. They let you define a set of functions that different data types can implement, so the same function call works correctly for many types.

Think of them as **interfaces** (or type classes in Haskell / traits in Rust), but with Elixir's dynamic nature and pattern-matching style.

#	Concept	Meaning
1	Protocol	Declares what functions must be available
2	Implementation (defimpl)	Provides the actual code for a concrete type
3	Dispatch	At runtime Elixir looks at the data's type and calls the matching implementation
4	Fallback / Any	A default implementation for types that don't have a specific one
5	Built-in protocols	Enumerable, Collectable, Inspect, String.Chars, List.Chars, Access, etc.

A2

Define, implement, use

One protocol — Size — for four built-in types

1. DEFINING THE PROTOCOL

```
defprotocol Size do
  @doc "Calculates the size of a data structure"
  def size(data)
end
```

2. IMPLEMENTING IT

```
defimpl Size, for: BitString do
  def size(string), do: byte_size(string)
end

defimpl Size, for: Map do
  def size(map), do: map_size(map)
end

defimpl Size, for: List do
  def size(list), do: length(list)
end

defimpl Size, for: Tuple do
  def size(tuple), do: tuple_size(tuple)
end
```

3. USING IT · REAL

```
Size.size("hello")      # => 5
Size.size(%{a: 1, b: 2}) # => 2
Size.size([1, 2, 3, 4]) # => 4
Size.size({1, 2, 3})    # => 3
```

NOTE One call, four implementations: Elixir looks at the **type of the first argument** and runs **Size.BitString**, **Size.Map**, **Size.List** or **Size.Tuple**.

A3

Fallback and structs

A default for everything else — and your own types

4. FALLBACK IMPLEMENTATION (ANY)

```
defimpl Size, for: Any do
  def size(_), do: 0      # or raise, or whatever default you want
end
```

You can also mark the protocol itself with **@fallback_to_any true** so unimplemented types automatically use the **Any** implementation.

MEASURED Both halves are needed. With only the **for: Any** implementation, **Size.impl_for(123)** is still **nil** and **Size.size(123)** still raises **protocol Size not implemented for Integer**. The fallback only kicks in once the protocol says **@fallback_to_any true**:

THE PROTOCOL OPTS IN · REAL

```
defprotocol Size2 do
  @fallback_to_any true
  def size(data)
end

defimpl Size2, for: List do
  def size(list), do: length(list)
end

defimpl Size2, for: Any do
  def size(_), do: 0
end

Size2.size(123)      # => 0
Size2.impl_for(123)  # => Size2.Any
Size2.size([1, 2])   # => 2      (a specific impl still wins)
```

5. A protocol for structs

IEX · REAL

```
defmodule User do
  defstruct [:name, :age]
end

defimpl Size, for: User do
  def size(%User{}), do: 2 # two fields
end

Size.size(%User{name: "Alice", age: 30}) # => 2
```

- A struct is a map underneath, but it does **not** fall back to the **Map** implementation. Before **defimpl Size, for: User**, the same call raised **protocol Size not implemented for User (a struct)**.
- Every struct gets its own implementation — here **Size.User**.

GOTCHA On Elixir 1.20, a script cannot use a struct at the top level of the same **.exs** file that defines it (**cannot access struct User ... in the same context that defines it**). Paste these examples into iex, or put the struct in its own file.

A4

Common built-in protocols

Enumerable, String.Chars and Inspect

Enumerable — used by Enum and Stream

IEX · REAL

```
Enum.map([1, 2, 3], &(&1 * 2)) # works because List implements Enumerable
# => [2, 4, 6]
Enum.map(%{a: 1, b: 2}, fn {k, v} -> {k, v * 2} end)
# => [a: 2, b: 4]
```

String.Chars — used by to_string/1 and string interpolation

IEX · REAL

```
defimpl String.Chars, for: User do
  def to_string(%User{name: name, age: age}) do
    "#{name} (#{age})"
  end
end

"Hello #{%User{name: "Bob", age: 25}}" # => "Hello Bob (25)"
```

Inspect — controls how data appears in the console / inspect/1

IEX · REAL

```
defimpl Inspect, for: User do
  def inspect(%User{name: name, age: age}, _opts) do
    "#User<#{name}, #{age}>"
  end
end

%User{name: "Bob", age: 25}
# before the defimpl: %User{name: "Bob", age: 25}
# after:              #User<Bob, 25>
```

A5

Deriving and checking

@derive for structs; impl_for/1 to ask who implements

7. DERIVING PROTOCOLS (CONVENIENT FOR STRUCTS)

```
defmodule Point do
  @derive [Inspect, Size] # if Size is already defined for maps/structs
  defstruct [:x, :y]
end
```

CORRECTION The comment in the notes does not hold. **@derive Size** copies the protocol's **Any** implementation into the struct — so it needs **defimpl Size, for: Any**. With only the Map implementation, compiling **Point** fails: **could not load module Size.Any ... cannot derive Size for Point**.

WITH DEFIMPL SIZE, FOR: ANY IN PLACE · REAL

```
Size.size(%Point{x: 1, y: 2})    # => 0           (the Any code)
Size.impl_for(%Point{x: 1, y: 2}) # => Size.Point  (a real impl for Point)
%Point{x: 1, y: 2}              # => %Point{x: 1, y: 2}
```

- A bare **@derive Inspect** prints the struct exactly as before. Deriving pays off with options: **@derive {Inspect, only: [:x]}** prints **#Point<x: 1, ...>** (measured).

8. Checking whether a type implements a protocol

IEX · REAL

```
Size.impl_for("hello")    # => Size.BitString
Size.impl_for(%{})        # => Size.Map
Size.impl_for(123)        # => nil (or Size.Any if fallback is enabled)
```

A6

Practical mini-example: a Blank protocol

What counts as “blank” differs by type — a natural fit for a protocol

BLANK.EX

```
defprotocol Blank do
  @doc "Returns true if the data is considered blank"
  def blank?(data)
end

defimpl Blank, for: BitString do
  def blank?(""), do: true
  def blank?(_), do: false
end

defimpl Blank, for: List do
  def blank?([]), do: true
  def blank?(_), do: false
end

defimpl Blank, for: Map do
  def blank?(map), do: map_size(map) == 0
end

defimpl Blank, for: Atom do
  def blank?(nil), do: true
  def blank?(false), do: true
  def blank?(_), do: false
end
```

USAGE · REAL

```
Blank.blank?("")      # true
Blank.blank?("hi")    # false
Blank.blank?([])      # true
Blank.blank?(%{})     # true
Blank.blank?(nil)     # true
Blank.blank?(0)       # error (no implementation) unless you add Any
# ** (Protocol.UndefinedError) protocol Blank not implemented for Integer
```

Protocol takeaways

- Protocols give you **open polymorphism** — you can add implementations for types you don't own.
- Dispatch is based on the **concrete type** of the first argument.
- Prefer protocols when you need the same behaviour across many different data shapes.
- Prefer behaviours (**@behaviour**) when you want modules to implement a known set of callbacks (more OOP-style) — Part B.

PART B

Behaviours

A contract between modules, checked by the compiler

B1

Behaviours — core concepts

“This module must implement these functions”

Elixir behaviours define a contract that a module must fulfil. They specify a set of functions (callbacks) that any implementing module is expected to provide.

They are the primary way Elixir expresses “this module must implement these functions,” and they power most of OTP — **GenServer**, **Supervisor**, **Application** and more (B5 lists which ones really are behaviours).

Concept	Meaning
Behaviour	A module that declares required (and optional) callbacks using @callback
Callback	A function specification that implementing modules must (or may) define
@behaviour	Attribute a module uses to declare that it implements a behaviour
Compiler check	Elixir warns at compile time if required callbacks are missing
Optional callbacks	Declared with @optional_callbacks — implementing them is not mandatory

Behaviour

- A contract between **modules**
- Compile-time, static

Protocol

- Polymorphism over **data types**
- Runtime dispatch

B2

Defining a behaviour

Parser: two required callbacks and one optional

PARSER.EX

```
defmodule Parser do
  @moduledoc "A behaviour for parsing data into a known format"

  @doc "Parses the given input and returns {:ok, result} or {:error, reason}"
  @callback parse(input :: term()) :: {:ok, term()} | {:error, term()}

  @doc "Returns a human-readable name for this parser"
  @callback name() :: String.t()

  # Optional callback
  @callback validate(input :: term()) :: boolean()
  @optional_callbacks validate: 1
end
```

WHAT THE COMPILER RECORDED · REAL

```
Parser.behaviour_info(:callbacks)      # => [validate: 1, parse: 1, name: 0]
Parser.behaviour_info(:optional_callbacks) # => [validate: 1]
```

B3

Reading a @callback line

What `::` and `term()` mean, piece by piece

THE LINE

```
@callback perform(state :: term()) :: {:ok, new_state :: term()} | {:error, reason :: term()}
```

What is `::`?

- `::` is the **typespec operator**: “this thing has the following type”.
- **Left** of `::` → the name of the argument or the return value.
- **Right** of `::` → the type.

What is `term()`?

- The **most general type** in Elixir: **any Elixir value**.
- Integers, floats, atoms, strings, lists, maps, tuples, pids, functions, structs... everything is a term.
- **state :: term()** → “I accept **any** value as **state**”.

Breaking the whole line down

Piece	Meaning
@callback	A required function that modules implementing the behaviour must define
perform(...)	The function name and its parameters
state :: term()	First (and only) argument is named state and can be any value
:: (after the parameters)	“This function returns...”
{:ok, new_state :: term()}	Success case: a tuple with :ok and a new state (any value)
	or
{:error, reason :: term()}	Error case: a tuple with :error and a reason (any value)

More precise types (better style)

SPECIFIC TYPES INSTEAD OF TERM()

```
@callback perform(state :: map()) ::
  {:ok, new_state :: map()} |
  {:error, reason :: atom() | String.t()}
```

OR WITH CUSTOM TYPES

```
@type state :: map()
@type reason :: atom() | String.t()

@callback perform(state) :: {:ok, state} | {:error, reason}
```

VERIFIED All three forms compile to the same callback, **perform/1** (checked with **behaviour_info(:callbacks)**). The precise ones document more and let tools like Dialyzer catch more.

Quick summary

Symbol / word	Meaning
::	"has the type"
term()	any Elixir value
@callback	declares a required function for the behaviour

B4

Implementing and using a behaviour

Two parsers, one runner that trusts the contract

JSON_PARSER.EX

```
defmodule JSONParser do
  @behaviour Parser

  @impl true
  def parse(input) when is_binary(input) do
    case Jason.decode(input) do
      {:ok, data} -> {:ok, data}
      {:error, reason} -> {:error, reason}
    end
  end

  def parse(_, do: {:error, :invalid_input})

  @impl true
  def name, do: "JSON Parser"

  # Optional callback - we choose to implement it
  @impl true
  def validate(input) when is_binary(input), do: String.trim(input) != ""
  def validate(_, do: false)
end
```

DEPENDENCY Jason is a Hex package, not part of Elixir: add `{:jason, "~> 1.4"}` to `deps`, or `Mix.install([{:jason, "~> 1.4"}])` in a script. Since Elixir 1.18 the built-in `JSON.decode/1` returns the same `{:ok, map}` here.

CSV_PARSER.EX

```
defmodule CSVParser do
  @behaviour Parser

  @impl true
  def parse(input) when is_binary(input) do
    rows =
      input
      |> String.split("\n", trim: true)
      |> Enum.map(&String.split(&1, ","))

    {:ok, rows}
  end

  def parse(_), do: {:error, :invalid_input}

  @impl true
  def name, do: "CSV Parser"

  # We skip the optional `validate/1` - no warning
end
```

3. USING THE BEHAVIOUR

```
defmodule ParserRunner do
  def run(parser_module, input) do
    # We can rely on the contract
    IO.puts("Using: #{parser_module.name()}")

    case parser_module.parse(input) do
      {:ok, result} ->
        IO.inspect(result, label: "Parsed")
        :ok

      {:error, reason} ->
        IO.puts("Failed: #{inspect(reason)}")
        :error
    end
  end

  # Usage
  ParserRunner.run(JSONParser, ~s({"name": "Alice", "age": 30}))
  ParserRunner.run(CSVParser, "name,age\nAlice,30\nBob,25")
end
```

REAL OUTPUT · ELIXIR 1.20.4 / OTP 29

```
Using: JSON Parser
Parsed: %{"age" => 30, "name" => "Alice"}
Using: CSV Parser
Parsed: [{"name", "age"}, ["Alice", "30"], ["Bob", "25"]]
```

- Both calls return **:ok**. **ParserRunner.run(JSONParser, "{bad}")** prints **Failed: %Jason.DecodeError{position: 1, token: nil, data: "{bad}"}** and returns **:error**.
- **CSVParser** skips the optional **validate/1** and compiles with **no warning**.

The compiler check, for real

DEFMODULE BROKENPARSER — @BEHAVIOUR PARSE, ONLY PARSE/1 · REAL WARNING

```
warning: function name/0 required by behaviour Parser is not implemented
      (in module BrokenParser)
```

@IMPL TRUE ON A FUNCTION THAT IS NOT A CALLBACK · REAL WARNING

```
warning: got "@impl true" for function helper/0 but no behaviour specifies
      such callback. The known callbacks are:
  * Parser.name/0 (function)
  * Parser.parse/1 (function)
  * Parser.validate/1 (function)
```

@IMPL ON PARSE/1 BUT NOT ON NAME/0 · REAL WARNING

```
warning: module attribute @impl was not set for function name/0 callback
      (specified in Parser). This either means you forgot to add the
      "@impl true" annotation before the definition or that you are
      accidentally overriding this callback
```

4. The @impl attribute (recommended)

THREE FORMS

```
@impl true      # tells the compiler this function implements a callback
@impl Parser    # more explicit - implements a callback from Parser
@impl false     # explicitly marks a function as *not* a callback
```

NOTE Using `@impl true` gives better compiler warnings — the last two warnings above only exist because of it — and makes the code self-documenting.

B5

A worker behaviour, and OTP behaviours

Your own contract — and the ones OTP gives you

5. A SIMPLE WORKER BEHAVIOUR

```
defmodule Worker do
  @moduledoc "Behaviour for background workers"

  @callback init(args :: term()) :: {:ok, state :: term()} | {:error, reason :: term()}
  @callback perform(state :: term()) :: {:ok, new_state :: term()} | {:error, reason :: term()}
  @callback terminate(reason :: term(), state :: term()) :: :ok
end

defmodule EmailWorker do
  @behaviour Worker

  @impl true
  def init(opts) do
    {:ok, %{queue: opts[:queue] || "default", retries: 0}}
  end

  @impl true
  def perform(state) do
    # pretend to send an email
    IO.puts("Sending email from queue: #{state.queue}")
    {:ok, %{state | retries: 0}}
  end

  @impl true
  def terminate(_reason, _state), do: :ok
end
```

IEX · REAL

```
{:ok, state} = EmailWorker.init([])
# => {:ok, %{queue: "default", retries: 0}}
EmailWorker.perform(state)
# Sending email from queue: default
# => {:ok, %{queue: "default", retries: 0}}
{:ok, state} = EmailWorker.init(queue: "emails")
EmailWorker.perform(state)
# Sending email from queue: emails
```

6. OTP behaviours — the most common use

MY_SERVER.EX

```
defmodule MyServer do
  use GenServer      # expands to @behaviour GenServer + default implementations

  # You only implement the callbacks you care about
  @impl true
  def init(initial_value) do
    {:ok, initial_value}
  end

  @impl true
  def handle_call(:get, _from, state) do
    {:reply, state, state}
  end

  @impl true
  def handle_cast({:set, value}, _state) do
    {:noreply, value}
  end
end
```

IEX · REAL

```
{:ok, pid} = GenServer.start_link(MyServer, 1)
GenServer.call(pid, :get)           # => 1
GenServer.cast(pid, {:set, 42})
GenServer.call(pid, :get)           # => 42
MyServer.module_info(:attributes)[:behaviour]
# => [GenServer]
```

- **GenServer** declares **9 callbacks, 8 of them optional** — only **init/1** is required — which is why “you only implement the callbacks you care about”.

Other important OTP modules — which are behaviours?

Module	Behaviour?	Note
Supervisor	Yes	You implement init/1
Application	Yes	You implement start/2
GenServer	Yes	init/1 plus the handle_* callbacks you need
DynamicSupervisor	Yes	You implement init/1
Agent	No	A ready-made process (built on GenServer) — you call it, not implement it
Task	No	A module of functions (start, async, await)
Registry	No	A module of functions for naming processes

CORRECTION The notes list all seven as OTP behaviours. Measured on 1.20.4 with `function_exported?(mod, :behaviour_info, 1)`: only **Supervisor**, **Application**, **GenServer** and **DynamicSupervisor** define callbacks. Agent, Task and Registry are plain modules.

B6

Runtime checks, best practices, recap

And behaviour vs protocol, side by side

7. CHECKING IMPLEMENTATIONS AT RUNTIME · REAL

```
# Returns true if the module implements the behaviour
function_exported?(JSONParser, :parse, 1) # true

# Or more precisely:
Code.ensure_loaded?(JSONParser) &&
  function_exported?(JSONParser, :parse, 1) # true
```

PRECISION `function_exported?/3` proves that a **function exists** — not that the module signed the contract: `function_exported?(Map, :get, 2)` is **true** too. To see the declared behaviours, read the module's attributes: `JSONParser.module_info(:attributes)[:behaviour] → [Parser]` (and **nil** for `Map`).

8. Best practices

- Always use `@impl true` (or `@impl BehaviourName`) on callback implementations.
- Prefer `@optional_callbacks` for functions that are nice-to-have.
- Document every `@callback` with `@doc`.
- Keep the behaviour focused — one clear responsibility.
- When you control both the behaviour and the implementations, consider whether a simple module + pattern matching would be clearer.

Quick comparison recap

Feature	Behaviour	Protocol
Target	Modules	Data types
Dispatch	Compile-time / explicit module	Runtime (based on data type)
Common use	OTP, plugins, strategies	Enumerable, Inspect, String.Chars
Missing impl	Compiler warning	Runtime error (or fallback)

VERIFIED Both “missing impl” cells are measured: a missing `name/0` gave the compiler warning in B4, and `Blank.blank?(0)` raised `Protocol.UndefinedError` in A6.